Plan: make `mobile_vehicle_counter_pi` conform to `README.md`

Status: **draft for review. No code has been changed yet.**

`README.md` is the specification. This plan describes how to turn the current project, which is a copy of the collector server, into the Pi client the README describes.

1. Current state

The project is still a copy of `mobile_vehicle_counter_server`:

- **Server code present:** the axum collector (`src/collector_server.rs`, route `/api/alpr/events`, `process_event`), the frontend API (`src/api_server.rs`, `src/handlers/`, JWT login), a sea-orm schema (`src/entities/`, `migration/`), `src/time.rs`, and server deploy files.
- **Missing, and required by the README:**
  - A local SQLite store with `pending`, `delivered` and `failed` statuses.
  - A sender that POSTs to the remote collector and verifies the receipt.
  - Retry with backoff, and the error-to-status table.
- **Already usable:** `Cargo.toml` already lists `rusqlite` (bundled), `sha2`, `hex`, `uuid` (v4), `base64`, `serde`, `serde_json`, `axum`, `tokio` and `dotenvy`. `collector_server.rs` shows the exact wire format and the `EventAck` shape the Pi must read.

2. Decisions (settled)

#	Topic	Decision
1	Camera input	The ALPR camera POSTs JSON to `/api/alpr/events` on the Pi. The structure is the collector's payload without `event_id`: `camid`, `date`, `plate`, `image`, `plate_image`.
2	Camera authentication	None. Cameras and the Pi are alone on a LAN.
3	Oversize bodies / rejecting cameras	The Pi **never rejects** a camera post, because cameras cannot retry. See section 5.
4	Retention	No code clears image data. The operator does it manually.
5	Naming	Crate `mobile_vehicle_counter_pi`, binary `run_pi_client`.

3. Strip out server-only code

Delete

- `src/api_server.rs`
- `src/handlers/`
- `src/entities/`
- `src/time.rs` (the server normalises dates, the Pi does not)
- `src/db.rs` (replaced by `store.rs`)
- `src/bin/run_frontend_api.rs`
- `src/bin/run_data_collector.rs`
- `src/bin/migrate_data.rs`
- `migration/`
- `tests/location_test.rs`, `tests/sublocations_test.rs`, `tests/collector_test.rs`
- `http_tests/login.http`
- `deploy/nginx-collector.conf`
- Stale directories: `.idea/`, `images/`, `out/`, `server_db/`
- Stale files: `README.pdf`, `mermaid-filter.err`

`Cargo.toml`

- Rename the package to `mobile_vehicle_counter_pi`. Set `default-run = "run_pi_client"`.
- Remove: `sea-orm`, `migration`, `jsonwebtoken`, `argon2`, `tower-http`, `chrono`, `tower` (unless the tests need it).
- Add: `reqwest` (`rustls-tls`, `json`).
- Keep: `rusqlite`, `uuid`, `sha2`, `hex`, `base64`, `serde`, `serde_json`, `axum`, `tokio`, `dotenvy`.

`.env`

- Remove `JWT_SECRET`, `IMAGE_DIR`, `API_ADDR`, `COLLECTOR_ADDR`, `DATABASE_URL`.
- Add the settings in section 4.
- Do not commit real secrets. Provide `.env.example` and make sure `.env` is in `.gitignore` (the current `.env` is committed).

`TODO.md`

- Replace the server items with Pi items (graceful shutdown, alerting on `failed` rows, disk-full handling).

4. New structure, mapped to the README

Module	Responsibility	README section
`config.rs`	Reads `.env` / environment (below).	Configuration
`store.rs`	Opens the SQLite database, applies pragmas, creates the table, inserts events, selects due events, records outcomes.	Local event storage
`event.rs`	The six-field `Event`. Generates `event_id` with `Uuid::new_v4()` (the only place an id is created). Serialises **once** with `serde_json::to_vec`.	Request, `event_id`, same-bytes rule
`ingest.rs`	Axum handler for `POST /api/alpr/events` from cameras. See section 5.	Architecture steps 1-2
`outcome.rs`	Pure functions: classify a status code or error as `Delivered`, `Retry` or `Reject`; verify the receipt; lowercase-hex SHA-256.	Response, Errors table
`sender.rs`	Loop: pick the oldest due `pending` event, POST the stored bytes, classify, record. One event at a time.	Retry behaviour
`bin/run_pi_client.rs`	Starts the ingest server and the sender loop together.	Architecture

Settings

Variable	Meaning	Default
`COLLECTOR_URL`	Collector endpoint, `http://...` while testing, `https://...` later	none, required
`COLLECTOR_TOKEN`	Value for `X-Collector-Token`	none, required
`PI_DB_PATH`	Local SQLite file	`pi_events.db`
`PI_LISTEN_ADDR`	Where cameras POST	`0.0.0.0:3000` (LAN-facing)
`REQUEST_TIMEOUT_SECS`	Per-request timeout to the collector	`60`
`MAX_CAMERA_BODY_BYTES`	Cap on a camera POST (section 5)	`67108864` (64 MB)

Database schema

The README fields:

Column	Type	Notes
`event_id`	TEXT PRIMARY KEY	Random v4 UUID
`status`	TEXT NOT NULL	`pending`, `delivered`, `failed` (CHECK constraint)
`body`	BLOB NOT NULL	Exact request body bytes
`attempts`	INTEGER NOT NULL DEFAULT 0	
`created_at`	TEXT NOT NULL	UTC, set on insert
`delivered_at`	TEXT	Set on `delivered`

Extra columns (the README allows extras for querying):

Column	Why
`next_attempt_at`	The README says “oldest first” but also “back off” and “retry slowly” on `401`/`422`. With one global backoff, one stuck event would block all newer events. The sender picks the oldest event whose `next_attempt_at` has passed.
`last_error`	Last status code or error text, so an operator can see why an event is `pending` or `failed`.

Pragmas: `journal_mode = WAL`, `synchronous = FULL`. An index on `(status, next_attempt_at, created_at)` keeps the sender query cheap.

Schema versioning uses `PRAGMA user_version` with a small ordered list of SQL steps. No migration framework.

5. Ingest rules (camera side)

Cameras cannot retry, so the Pi never rejects a camera post because of its content.

1. **Acknowledge only after commit.** The Pi answers `200` once the row is committed to SQLite, and not before.
2. **Valid camera JSON** (the five fields as strings): add `event_id`, serialise once, store as `pending`.
3. **Anything else** (invalid JSON, a missing or non-string field): store the raw bytes under a freshly generated `event_id` with status `failed` and `last_error = "unparseable camera payload"`. It is never sent. It is kept for inspection. Note that the `body` of such a row is the camera’s raw bytes, not a collector request body.
4. **The `date` is not validated by the Pi.** A bad `date` is sent, the collector answers `400`, and the event becomes `failed` through the normal path.
5. **Size cap.** `MAX_CAMERA_BODY_BYTES` (default 64 MB) is well above the collector’s 20 MB, so an event that is too large for the collector is still stored and later fails there with `413`. Do not rely on axum’s default 2 MB limit. A post over the cap is the only content-independent rejection that remains (a `413` to the camera); it exists to stop a faulty camera filling the SD card.
6. **If the database write fails** (disk full, corrupt file), the Pi must return `5xx`, because it cannot honestly confirm the event is stored. The camera cannot retry, so the event is lost. This is logged loudly. Protection against this is keeping free disk space, which is the operator’s manual retention.

6. Sender behaviour (must match the README exactly)

- **Body:** send the stored bytes, unchanged, on every attempt. Never rebuild them from other columns. No compression. Headers: `Content-Type: application/json`, `X-Collector-Token`.
- **Hash:** SHA-256 over exactly the stored bytes, lowercase hex.
- **Delivered** only when all three hold: status `200`, the response `event_id` equals the stored `event_id`, and the response `content_hash` equals the local hash. The `event_id` is read back from the stored body.
- **`200` with a bad receipt** (wrong id, wrong hash, unreadable JSON): stays `pending`, logged loudly.

- **Status mapping**

Result	Action
`200` with a verified receipt	`delivered`
`400`, `403`, `404`, `409`, `413`, corrupt stored body	`failed`, alert
`401`, `422`	`pending`, slow backoff
`429`, `500`, `502`, `503`, `504`, timeout, connection error	`pending`, normal backoff
Any other status	`failed`, log it (the README lists none; the example treats it as unretryable)

- **Backoff:** exponential with jitter, 2 s, 4 s, 8 s and so on, capped at a few minutes (cap configurable, default 5 minutes). `401` and `422` start from the cap. The backoff state is per event (`next_attempt_at`) and survives reboot.
- **Request timeout:** 60 s default.
- **Concurrency:** one event at a time.
- **No deletes:** the code contains no `DELETE`. Only `status`, `attempts`, `delivered_at`, `next_attempt_at` and `last_error` change after insert.
- **Alerting:** README says "alert" for `failed`. Initial implementation: an `ERROR` log line per `failed` event. A real alert channel is a later decision.

7. Tests

Unit

- `outcome.rs`: one case per row of the errors table, and receipt mismatches (wrong id, wrong hash, unreadable JSON).
- Hash test vectors from the README (`abc`, empty string).
- `event.rs`: `event_id` is a valid v4 UUID, lowercase and hyphenated; serialising once and reading the id back gives the same id.

Store

- A committed event survives closing and reopening the database.
- A retry sends byte-identical bodies (compare hash before and after).
- The delivery path never deletes rows.
- Sender query returns the oldest due `pending` event, and skips events whose `next_attempt_at` is in the future.

Ingest (build the router and use `tower::ServiceExt::oneshot`)

- Valid JSON returns `200` and a `pending` row.
- Invalid JSON, or a missing field, returns `200` and a `failed` row holding the raw bytes.
- A body above 2 MB and below the cap is accepted.
- A body above the cap returns `413`.
- The reply is sent only after the row exists.

Sender, against a mock collector (a small axum server in the test)

- First delivery becomes `delivered`.
- A duplicate `200` also becomes `delivered`.
- A timeout followed by a retry sends the same bytes.
- `409` becomes `failed`. `422` stays `pending`. `401` stays `pending`. A bad receipt stays `pending`.

Contract (manual or CI)

- Run the server project's real collector against a temporary database and deliver several events, including a duplicate and an unknown camera. This tests the README's claims end to end and needs both crates.

8. Deployment and documentation

- Replace `deploy/mobile-vehicle-collector.service` with a Pi unit for `run_pi_client`: `Restart=on-failure`, a dedicated user, write access only to the database directory, and `.env` from the working directory.
- Update `README.md` after implementation:
  - Remove the “leftovers from the server project” sentence.
  - Add the camera-facing input (section 2, decision 1).
  - Describe ingest-time `failed` rows and the “never reject the camera” behaviour (section 5).
  - Note the extra columns `next_attempt_at` and `last_error`.
- In the server project, `TODO.md` and `deploy/nginx-collector.conf` still refer to `README_PI.md`. Decide whether this README becomes the single source for the protocol.

9. Order of work

1. Confirm the project is under git, so deletions can be undone.
2. Strip the server code and fix `Cargo.toml` until the empty crate builds.
3. `event.rs` and `outcome.rs` with unit tests (pure, and cover most of the contract).
4. `store.rs` with tests.
5. `sender.rs` with mock-collector tests.
6. `ingest.rs` and `bin/run_pi_client.rs`.
7. Contract test against the real collector.
8. Deploy unit, README update, `TODO.md`.

Each step should leave `cargo build` and `cargo test` passing before the next begins.

10. Open points for review

- **Unknown HTTP statuses:** the plan marks any status not in the README table as `failed`. Alternative: treat them as `pending`, so nothing is ever given up on silently.
- **Alerting:** log-only for now.
- **Camera timestamp format:** the plan assumes the camera already sends RFC 3339 with an offset. If it does not (for example a local time without an offset), the Pi needs a conversion step with a configured time zone. Please confirm with a sample camera payload.
- **Port:** the default listen port `3000` is a guess; tell me the real one the cameras are configured with.
- **Collector `camid` mapping:** `camid` must match `camera.id` on the server. The Pi sends the camera’s value unchanged; confirm the cameras’ ids are registered on the server.